PART TWO • CHAPTER 07

Thread pools, *or how to share a bounded crew.*

From `threading.Thread` *to* `concurrent.futures` *— when a pool helps, and when it just hides the problem.*

TOPIC	LANGUAGE	RUNTIME
Thread pools & futures	Python 3.12	75 min read

IN THIS CHAPTER

A thread pool is a *bounded crew* of workers that take jobs from a shared queue. You hand it a function and its arguments; it hands back a **future** — a promise that the answer will be ready later. Pools solve two problems at once: they cap how many threads exist, and they remove the cost of starting a new one for every task.

This chapter assumes you have seen `threading.Thread` but have not yet reached Ch. 8 · asyncio. We stay deliberately in the `concurrent.futures` module, which is the right tool for the overwhelming majority of I/O-bound Python programs.[1]

What you will learn

Pool	A fixed-size set of worker threads that consume a work queue.
Future	An object representing a computation that may not yet have completed.
Executor	The object you submit work to; it owns the pool and the queue.
GIL	The Global Interpreter Lock; only one thread runs Python bytecode at a time.

A note on scope

We cover *thread* pools specifically. Process pools (`ProcessPoolExecutor`) are touched on in 7.4 only to contrast sizing rules. ~~Async pools~~ are deferred to chapter 8.

7.1 · THREADS & THE GIL

Why a pool, and why bounded.

7.1

Spawning a thread in Python is cheap but not free. Each thread carries an OS-level stack (8 MB on Linux by default), a bookkeeping structure in the interpreter, and contention for the **GIL**. Unbounded spawning is the single most common cause of a Python server going sideways under load.

7.1.1 Three reasons to pool

1. **Bound memory.** A fixed worker count caps stack usage to a predictable multiple of the stack size.
2. **Amortize startup.** Thread creation is ~100 µs; reusing a worker for a 1 ms task matters.
3. **Backpressure for free.** When the queue fills, submitters block — the pool refuses to paper over a too-slow consumer.

7.1.2 When threads do *not* help

- Pure CPU work in pure Python — the GIL serializes it.
  - Use `ProcessPoolExecutor` instead.
  - Or drop into C via NumPy / `numba`.
- Work that already releases the GIL (e.g. a `requests` call) benefits regardless.
- Work that calls into a C extension holding a lock of its own.

> A thread pool is a queue in a trench coat. Everything interesting is in how the queue behaves when it is full, and in what the workers do when the queue is empty.

7.1.3 Rules of thumb

- [x] Is the workload I/O-bound?
- [x] Am I willing to cap memory with a worker count?
- [ ] Can I tolerate out-of-order completion?

THE GIL IN 3.13

PEP 703 introduces a no-GIL build. Until it is the default, reason as if the GIL is there.

STACK SIZE

Tunable via `threading.stack_size()` — rarely worth doing.

7.3 · SUBMITTING & COLLECTING WORK

The minimal executor.

7.3

The standard library ships `concurrent.futures.ThreadPoolExecutor`. It is a context manager; entering it spins the workers, exiting it joins them.

fetch_all.py — Python 3.12

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
import requests

urls = ["https://example.com/", "https://python.org/"]

def fetch(url: str) -> int:
    return requests.get(url, timeout=5).status_code

with ThreadPoolExecutor(max_workers=8) as ex:
    futures = {ex.submit(fetch, u): u for u in urls}
    for f in as_completed(futures):
        print(futures[f], f.result())
```

CONTEXT MANAGER

Exiting the `with` block calls `shutdown(wait=True)`.

NOTE

Use `as_completed` when order does not matter — results arrive in finish order, not submit order. For submit order, iterate `ex.map` instead.

TIP

Wrap `f.result()` in `try/except`. Exceptions raised inside a worker are deferred until you ask for the result.

WARNING

Never `submit()` from inside a worker of the same pool — you can deadlock. Use a separate pool, or just call the function directly.

DANGER

Do not share an unsynchronised mutable (a `list`, `dict`) across workers. The GIL protects bytecode, not your invariants.

SIZING · I/O-BOUND VS CPU-BOUND

7.4 · SIZING THE POOL

How many workers?

7.4

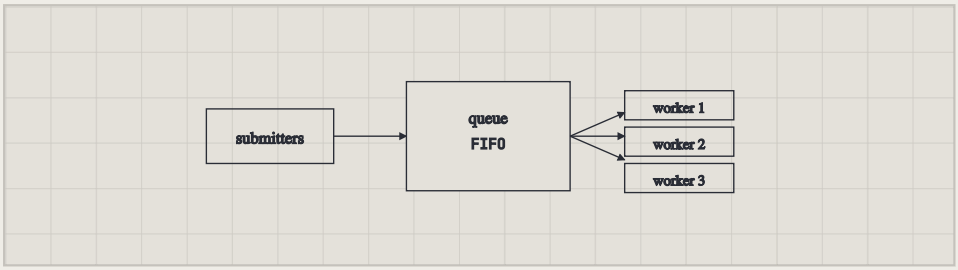

Fig. 7.1 Submitters push callables into a FIFO work queue; a fixed set of workers pull from it.

LITTLE'S LAW

Kleinrock 1975; applies to any stable queueing system.

A reasonable default for I/O-bound work is given by Little's law:

$$N = \lambda \cdot W \tag{7.1}$$

where N is the pool size, λ the arrival rate of requests, and W the average time a worker spends per request (mostly blocked on I/O).

Workload	Good default	Ceiling	Why
Local disk I/O	`4 – 8`	`32`	`Kernel queue depth`
HTTP calls	`8 – 32`	`256`	`Remote capacity`
DNS lookups	`16`	`64`	`Resolver cache`
Pure Python CPU	`1`	`1`	`GIL`

7.5 · EXERCISES

Work these before 7.6.

7.5

Model solutions are in Appendix C, pp. 342–346.

01 *Warm-up* SUBMIT / RESULT

Using `ThreadPoolExecutor`, compute the length of ten URLs in parallel and print them in *submission* order, not completion order.

02 *Sizing* LITTLE'S LAW

A service receives 40 requests/second; each spends 0.6 s blocked on a downstream API. Compute the pool size from equation *(7.1)*. Justify rounding up.

03 *Trap* DEADLOCK

Construct a program where a worker in pool *A* submits to the same pool and waits on the result. Show the deadlock; fix it with two pools.

> *Concurrency is a way of organising code. Parallelism is a property of how it runs.*
>
> — ROB PIKE · CONCURRENCY IS NOT PARALLELISM (2012)

NOTES

1. For CPU-bound work, substitute `ProcessPoolExecutor`; the API is identical, the cost model is not.
2. The `as_completed` iterator accepts a `timeout=` keyword.

SEE ALSO

Ch. 8 · asyncio; Ch. 9 · process pools.